

DOCUMENTO EXECUTIVO

Assistente Virtual de Dados Multiagêntico

Text-to-SQL agêntico sobre LangGraph + Gemini
Desafio Técnico — FRANQ

Leandro Henrique da Silva

Engenheiro de Controle e Automação

+55 (47) 9.8906-7677

dasilva.leandrohenrique1991@gmail.com

Sumário

Para numerar as páginas abaixo, clique com o botão direito sobre o sumário e escolha “Atualizar campo” (ou pressione F9). O Word pode também perguntar isso ao abrir o documento.

Sumário	2
1. Sumário executivo.....	5
A narrativa-mestre	5
2. As quatro capacidades avaliadas e como o código garante cada uma.....	6
2.1. Descoberta dinâmica de schema (proibido hardcoded)	6
2.2. Raciocínio multi-passo.....	6
2.3. Autocorreção	6
2.4. Apresentação adaptativa + transparência.....	6
3. A simetria entre os dois desafios.....	7
4. Conclusões do estudo do banco de dados.....	7
5. O grafo de agentes: diagrama e leitura do fluxo	10
O caminho principal.....	11
Os desvios e os loops	11
6. O que é um agente — e por que o nosso é.....	11
A gradação honesta	11
O veredito, e a ressalva que vale ouro	12
7. Os agentes (LLM)	13
7.1. Analista de Negócios	13
7.2. Engenheiro de Dados (text-to-SQL).....	14
7.3. Auditor de Dados (autocorreção semântica, nível 2)	15
7.4. Designer de Visualização	17
7.5. O Diretor (segunda função do Analista).....	18
8. Os componentes determinísticos (Python, não-LLM).....	19
8.1. Perfilador do Banco	19
8.2. Guardrail SQL.....	20
8.3. Guardrail de Visualização (interno ao nó visualizador)	20
8.4. Executor Somente-Leitura	20
9. A lógica dos fluxos e os contratos em ação.....	20
Um exemplo concreto.....	21
10. O dossiê do banco (schema enriquecido, gerado dinamicamente)	22

As 6 camadas.....	22
O Perfilador é um termômetro: dinâmico sem ser inteligente.....	22
Trecho real dos alertas (banco do desafio).....	22
11. Escalabilidade do dossiê.....	23
O que quebra num banco gigante	23
Os três movimentos da migração	23
12. Três cenários de arquitetura: o piso, o teto e o ponto ótimo	24
A — Chain fixa (o piso).....	24
E — Multi-agente puro com supervisor-LLM (o teto, o excesso).....	24
D — Grafo estruturado custom (o nosso, o ponto ótimo).....	24
13. Por que arestas condicionais, e não um supervisor-LLM	25
14. Segurança.....	25
Defesa em profundidade	25
15. Visualização: dois autores, um renderizador	25
Modo agêntico (o escolhido como padrão)	26
Modo pre_setado (a alternativa determinística).....	26
Outliers em destaque	26
O limite honesto do ECharts.....	26
16. Avaliação de modelos de mercado	26
Por que Gemini neste projeto	27
Benchmark do nosso Engenheiro.....	27
17. Escalabilidade, custo, robustez, latência e testabilidade	27
Escalabilidade	27
Custo.....	27
Robustez	28
Latência interativa	28
Testabilidade e depurabilidade	28
18. Metodologia de desenvolvimento	29
18.1. SDD — Spec-Driven Development	29
18.2. TDD — Test-Driven Development orientado a risco	29
18.3. Orientação a análise de critério	29
18.4. Orientação a revisão.....	29
18.5. Os cinco marcos.....	29

19. Organização sistêmica do código.....	30
19.1. Tabela dos módulos e suas interações.....	30
19.2. Assinaturas-chave por arquivo	31
19.3. A lógica da estrutura de pastas	31
20. Os testes.....	32
O que os testes garantiram	32
21. Sobre o LangGraph Studio: dossiê e diagnóstico.....	33
O que foi comprovado	33
O diagnóstico honesto da falha visual	33
22. Heurísticas de fallback (degradação graciosa).....	33
23. Limitações.....	34
De infraestrutura e escala.....	34
Da lógica dos agentes	34
Dos componentes determinísticos.....	34
24. Sugestões de melhorias e próximas versões	34
25. Encerramento	35

1. Sumário executivo

Este documento descreve a arquitetura e as decisões de engenharia do **Assistente Virtual de Dados — Multiagêntico**, um sistema **text-to-SQL agêntico** construído sobre LangGraph e Gemini. O usuário pergunta em linguagem natural; uma equipe de agentes especializados planeja a consulta, escreve o SQL, executa-o contra um banco SQLite, audita se o resultado realmente responde à pergunta e devolve uma resposta de negócio acompanhada do gráfico mais adequado — expondo, a cada passo, o raciocínio que levou até ali.

O sistema cumpre as quatro capacidades exigidas pelo enunciado — descoberta dinâmica de schema, raciocínio multi-passo, autocorreção e apresentação adaptativa com transparência — e vai além delas em segurança, robustez e observabilidade. A suíte de testes, orientada a risco, cobre os pontos vulneráveis do sistema.

A narrativa-mestre

Este desafio é o inverso conceitual de um pipeline de extração de documentos, e essa simetria orienta todas as decisões:

No Desafio 2 a incerteza estava nos dados, e usei um pipeline com LLM em pontos fixos; no Desafio 1 a incerteza está na pergunta e no caminho, então inverti — dei agência ao LLM onde ela é necessária, mantendo determinismo onde ela é perigosa.

Esse é o eixo do projeto: **agência onde há incerteza, determinismo onde há mecânica**. Interpretar uma pergunta ambígua, escrever SQL, julgar se um resultado responde ao que foi pedido e desenhar um gráfico são tarefas de incerteza semântica — território do LLM. Rotear o fluxo, validar a segurança de uma consulta, executá-la e impor tetos são mecânica — território do código determinístico.

2. As quatro capacidades avaliadas e como o código garante cada uma

O enunciado define quatro capacidades. A seguir, como cada uma é cumprida no código do projeto final.

2.1. Descoberta dinâmica de schema (proibido `hardcode`)

O primeiro nó do grafo é o **Perfilador** (`perfilador.py`), um componente determinístico que lê o banco em tempo de execução via `PRAGMA table_info`, `PRAGMA foreign_key_list` e `sqlite_master`. **Não há um único nome de tabela ou coluna escrito no código** — uma varredura dos fontes confirma isso por teste. Aponte o sistema para outro SQLite e o dossiê se reconstrói sozinho. É o que torna possível atender literalmente ao requisito de *entender a estrutura do banco dinamicamente*.

2.2. Raciocínio multi-passo

O **Analista** decompõe a pergunta em um plano de 1..N passos (`AnaliseDaPergunta.passos`). O grafo executa cada passo em sequência, mantendo um cursor (`indice_passo_atual`) e disponibilizando os resultados já obtidos (`resultados_anteriores`) para o passo seguinte. Perguntas que exigem etapas intermediárias — uma agregação que depende de um filtro calculado antes — são resolvidas encadeando passos, não forçando um único SQL monolítico.

2.3. Autocorreção

Há três laços de correção, todos visíveis no trace:

- **Sintático (nível 1):** se o SQL falha no SQLite ou é barrado pelo Guardrail, o erro real volta ao Engenheiro, que regenera a consulta com o motivo no prompt.
- **Semântico (nível 2):** o Auditor pode aprovar um SQL válido e ainda assim reprová-lo por não responder à pergunta — devolvendo ao Engenheiro com uma instrução de correção. É o que separa um sistema pleno de um júnior.
- **Com o humano (interrupt):** diante de ambiguidade material, o Analista pausa o grafo e pergunta ao usuário; a resposta vira premissa e o plano é refeito.

Cada laço tem um teto determinístico (ver seção 9), de modo que a autocorreção nunca se torna um loop infinito.

2.4. Apresentação adaptativa + transparência

A apresentação se adapta à pergunta: o sistema escolhe entre **gráfico** (autorado pelo Designer em ECharts) e **tabela** (forçada quando o usuário pede explicitamente, ou como porto seguro). A transparência é estrutural: o **trace ao vivo** mostra cada nó conforme executa; as **consultas SQL** ficam disponíveis na tela; as **premissas** assumidas são declaradas; e um **diagrama do caminho** percorrido acompanha cada resposta. O estado do grafo carrega plano, queries, erros e correções — a interface apenas os expõe.

3. A simetria entre os dois desafios

A inversão conceitual entre um pipeline de extração (Desafio 2) e este agente (Desafio 1) resume a maturidade arquitetural pretendida:

	Desafio 2 — pipeline	Desafio 1 — agente
Natureza	Fluxo fixo; LLM em pontos determinados	O LLM decide o fluxo, com loops e decisões em runtime
Direção	Documento não estruturado → dado estruturado	Pergunta em linguagem natural → consulta → resposta
Onde mora a incerteza	Nos dados (layouts, inconsistências)	Na pergunta (ambiguidade) e no caminho (qual SQL)
Controle	Máximo (grafo determinístico)	Agência concedida ao LLM, sob guardrails

Em ambos, o princípio é o mesmo — separar o que é semântico (LLM) do que é mecânico (Python). Muda apenas onde a fronteira é traçada, porque muda onde a incerteza vive.

4. Conclusões do estudo do banco de dados

A primeira atividade do projeto, antes de qualquer linha de arquitetura, foi inspecionar empiricamente o banco real (`anexo_desafio_1.db`): schema, amostras, formatos, domínios e volumetria. As conclusões abaixo moldaram decisões concretas do sistema.

Estrutura e integridade

- Quatro tabelas — `clientes` (100), `compras` (946), `suporte` (273), `campanhas_marketing` (248) — todas ligadas a `clientes` por `cliente_id`. Sem nulos, sem registros órfãos, sem duplicatas, sem índices/views/triggers.

Colunas redundantes desatualizadas

- A tabela `clientes` traz `valor_total_gasto` e `data_ultima_compra`, que **divergem 100% dos dados transacionais**: a primeira não bate com `SUM(compras.valor)`; a segunda nunca coincide com o `MAX` das datas reais de compra/contato. São colunas denormalizadas desatualizadas de propósito. A fonte confiável é sempre a tabela transacional — e o sistema **detecta e expõe essa divergência** em vez de confiar no atalho.

O campo “canal” tem três significados

- A coluna `canal` existe em três tabelas com **domínios disjuntos**: em `compras` é App/Loja Física/Site; em `suporte` é Chat/Telefone/E-mail; em `campanhas_marketing` é SMS/WhatsApp/E-mail. Filtrar “E-mail” na tabela errada devolve vazio silencioso — por isso o contexto de valores por coluna é indispensável.

A âncora temporal é julho de 2025, não hoje

- Os dados cobrem de 2024-07-22 a 2025-07-22. “Maio” só existe em 2025; “último ano” deve ancorar em 2025-07-22, nunca na data do relógio. O sistema resolve janelas relativas contra o `MAX(data)` do próprio banco.

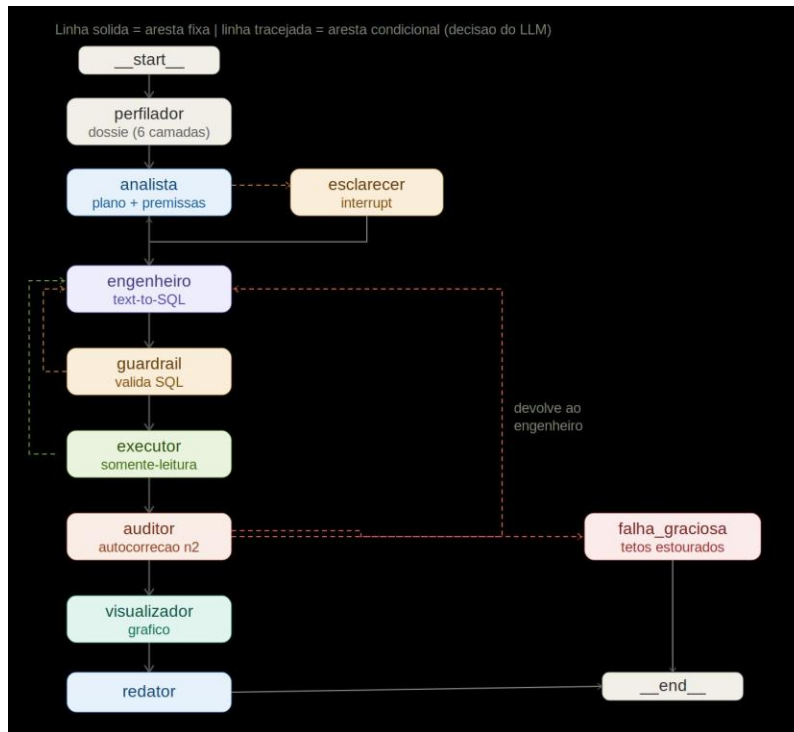
Formatos e cuidados de junção

- Datas em ISO (YYYY-MM-DD); booleanos como 0/1; estado por extenso (“São Paulo”, não “SP”). Dez clientes nunca acionaram o suporte — o que exige atenção a `INNER` vs `LEFT JOIN` em perguntas “por cliente”.

Por que isso importa para a avaliação: o enunciado insiste em descoberta dinâmica precisamente porque quem fixa o schema do PDF nem fica sabendo que as colunas-armadilha existem. Quem descobre dinamicamente precisa decidir em qual fonte confiar — e o nosso sistema decide pela fonte transacional, expondo a divergência com transparência.

5. O grafo de agentes: diagrama e leitura do fluxo

O diagrama abaixo é a topologia real do grafo compilado. Ele é a tese do projeto tornada visível.



Linha sólida = aresta fixa. Linha tracejada = aresta condicional (a decisão nasce em um agente). As arestas em amarelo mostarda são os retornos de autocorreção.

O caminho principal

Uma pergunta percorre, no caminho feliz: perfilador → analista → engenheiro → guardrail → executor → auditor → visualizador → diretor. O Perfilador monta o dossiê do banco; o Analista planeja; o Engenheiro escreve o SQL de cada passo; o Guardrail valida; o Executor roda em modo somente-leitura; o Auditor julga; o Visualizador desenha; e o Diretor redige a resposta final.

Os desvios e os loops

- **Ambiguidade** → analista → esclarecer → analista: pergunta material pausa o grafo (interrupt); a resposta do usuário vira premissa.
- **SQL inválido** → guardrail → engenheiro OU executor → engenheiro: o erro real volta para nova tentativa.
- **Resposta não satisfatória** → auditor → engenheiro: devolução semântica com instrução de correção.
- **Tetos estourados** → falha graciosa (exibida como “teto estourado”): qualquer limite atingido encerra com uma mensagem honesta, sem travar.

Quem **decide** esses desvios são os agentes, escrevendo campos no estado; as arestas apenas **leem** esses campos e roteiam. As arestas não pensam — registram. Por isso o diagrama distingue, pela cor da caixa, o que é agente (decide) do que é determinístico (executa).

6. O que é um agente — e por que o nosso é

Um avaliador atento pode perguntar: “uma chamada de LLM com um prompt de papel é mesmo um agente?” A resposta madura é que **o que torna um nó um agente não é o crachá, é o comportamento**: decisão, iteração e autoridade sobre o fluxo.

- O **Engenheiro** opera em loop: recebe o erro, decide a correção, tenta de novo.
- O **Auditor** tem poder de veto e devolução: reprova e manda o trabalho de volta.
- O **Analista** pode interromper o fluxo para perguntar ao usuário.
- O **Designer** toma decisões abertas: qualquer gráfico que a pergunta pedir.

Um nó que só transforma entrada em saída fixa — os determinísticos — não é agente, por mais prompt que tivesse.

A gradação honesta

Nem toda chamada de LLM é igualmente agêntica. O Engenheiro é o mais agêntico (vive no ciclo agir→observar→corrigir); Auditor e Designer têm agência de veto e de criação dentro de loops; a redação final é a **menos** agêntica — não tem loop nem toca o ambiente, só transforma resultados em texto. Por isso ela é modelada como **segunda função do Analista (o “Diretor” na exibição), não como um quinto agente**.

O veredito, e a ressalva que vale ouro

O teste decisivo é: quem decide o fluxo? As arestas só leem o estado; quem escreve `aprovado=False`, `precisa_esclarecimento=True` ou um SQL que falha e força `retry` são os LLMs. Somos, portanto, um **sistema multiagente com plano de controle determinístico**. Removemos deliberadamente a agência sobre a topologia (um supervisor-LLM escolhendo o próximo nó). Pelo crivo mais purista — “o LLM escolhe até a próxima etapa” — seríamos um *workflow agêntico*; e essa honestidade mostra que a escolha foi de engenharia, não de desconhecimento.

Os agentes decidem o quê e o se; o grafo só garante o onde e o até quando.

Disso deriva a regra de ouro de segurança: **a segurança mora na ferramenta, nunca no prompt do agente** — princípio detalhado na seção 14.

7. Os agentes (LLM)

Existe **um único LLM** no sistema — o Gemini que a fábrica `criar_llm()` instancia, a `temperature=0`. O que existe em plural são prompts de cargo: cada agente é o mesmo modelo sob um papel diferente, com um contrato Pydantic de saída via `with_structured_output`. É o par **prompt + contrato** que faz a decisão do modelo virar dado de estado roteável. A analogia: um único ator interpretando vários personagens — o personagem é o roteiro (prompt), não o ator (modelo).

7.1. Analista de Negócios

Interpreta e decompõe a pergunta do usuário em um plano de 1..N passos SQL, declara as premissas assumidas e decide se precisa esclarecer algo. É a porta de entrada da inteligência do sistema.

Existe porque o salto de “pergunta vaga em linguagem natural” para “consulta executável” é onde mora a maior incerteza semântica — e onde um plano explícito, com premissas declaradas, substitui o chute. A regra 00 do seu prompt resolve um problema real observado em fogo: pedidos de gráfico estatístico (“regressão polinomial”) não devem ser recusados, porque o tipo de gráfico é alçada do Designer, não do Analista.

Prompt de cargo (real, do código)

Placeholders entre parênteses e maiúsculas são preenchidos em tempo de execução (a pergunta, o dossiê, os resultados).

Você é um Analista de Negócios. Sua tarefa: decompor a pergunta do diretor em um plano de 1 a N consultas SQL (passos), declarar as premissas assumidas e decidir se precisa de esclarecimento.

REGRAS DE AMBIGUIDADE (siga à risca):

00. TIPO DE GRÁFICO NÃO É DA SUA ALÇADA – NUNCA RECUSE POR CAUSA DELE: você cuida dos DADOS, não da visualização. O tipo, o nome ou a técnica de gráfico (dispersão, regressão polinomial, tendência, heatmap, 3D, etc.) é decidido e construído por OUTRO agente (o Designer) mais adiante. Se a pergunta pede um gráfico e as colunas para os eixos/medidas EXISTEM no dossiê, então HÁ PLANO: monte os passos para BUSCAR esses dados (ex.: data, valor e canal de cada compra) e ignore a parte estatística/visual do pedido – ela será resolvida ou adaptada pelo Designer. É PROIBIDO marcar `pedido_fora_de_escopo` ou pedir esclarecimento alegando que um cálculo estatístico (regressão, suavização) ou um tipo de gráfico está 'além da capacidade de consulta': sua tarefa é só entregar as colunas certas.

0a. DADO INEXISTENTE → ESCLARECIMENTO ANCORADO NO SCHEMA, NUNCA RECUSA DIRETA: se a pergunta exige um dado que NÃO existe no dossiê (ex.: 'quantidade de itens' quando a tabela só tem o valor total), NÃO recuse e NÃO invente o dado – marque `precisa_esclarecimento=true` e faça UMA pergunta de esclarecimento que: (1) diga explicitamente que o dado não existe, citando as colunas REAIS da tabela; e (2) ofereça 2 a 3 alternativas CONCRETAS e úteis construídas SOMENTE com colunas que existem no dossiê (ex.: valor × canal, valor × data da compra). O usuário escolhe e a análise segue. Reserve `pedido_fora_de_escopo=true` APENAS para pedidos de ESCRITA (regra 0) ou quando NENHUMA coluna do banco permitir qualquer análise relacionada ao tema.

0b. PERGUNTA DE ESCLARECIMENTO ANCORADA NO SCHEMA: toda alternativa oferecida na pergunta de esclarecimento DEVE existir no dossiê (tabela + coluna + valores do dicionário). PROIBIDO sugerir dados inexistentes (ex.: 'categoria do produto' se não há tal coluna) – isso prende o usuário num ciclo de perguntas irrespondíveis.

0. PEDIDO DE ESCRITA NÃO É AMBIGUIDADE: se a pergunta pede ALTERAR dados (atualizar/inserir/apagar/criar – UPDATE/INSERT/DELETE/ALTER), NÃO peça esclarecimento e NÃO

ofereça alternativas: marque `pedido_fora_de_escopo=true` E `pedido_de_escrita=true`, com motivo curto. O sistema é SOMENTE LEITURA e responderá com a recusa honesta. (`pedido_de_escrita` fica FALSE em qualquer outra recusa – ele distingue a mensagem de segurança das demais.)

1. ANTES de declarar ambiguidade, verifique a ÂNCORA TEMPORAL e o dicionário de valores do dossiê. Ex.: um mês sem ano que só existe em UM ano na base NÃO é ambíguo – é premissa verificada (declare-a).
2. Janelas relativas ('último ano') ancoram na âncora temporal do dossiê – isso é premissa, não ambiguidade.
3. PERÍODO NÃO ESPECIFICADO não é ambiguidade material: assumo todo o histórico disponível e DECLARE a premissa (não pergunte).
4. Peça esclarecimento APENAS se a dúvida mudaria materialmente a resposta (ex.: a pergunta não diz qual métrica ou qual assunto).
5. Cada passo do plano tem UM objetivo claro e verificável. Prefira o menor número de passos que responda bem.

```
{bloco_esclarecimentos}{bloco_dialogo}
{dossie}

{pergunta_apresentada}
```

Contrato de saída (Pydantic)

```
class AnaliseDaPergunta(BaseModel):
    precisa_esclarecimento: bool
    pergunta_de_esclarecimento: str | None
    premissas: list[str]
    passos: list[PassoDoPlano]          # cada passo: objetivo: str
    pedido_fora_de_escopo: bool
    motivo_fora_de_escopo: str | None
    pedido_de_escrita: bool
```

Cada campo:

- `precisa_esclarecimento` — True apenas se a ambiguidade mudaria materialmente a resposta. É o que a aresta lê para decidir entre perguntar ao usuário e seguir.
- `pergunta_de_esclarecimento` — o texto da pergunta ao usuário, quando há ambiguidade material.
- `premissas` — tudo que foi assumido (ex.: “maio = 2025, único na base”) — vira transparência na resposta.
- `passos` — a lista ordenada de objetivos de consulta; é o plano multi-passo.
- `pedido_fora_de_escopo` / `motivo_fora_de_escopo` — marcam pedidos que o sistema não atende como consulta (ex.: alteração de dados); a aresta roteia direto para a recusa honesta.
- `pedido_de_escrita` — distingue um pedido de CRUD (escrita) de uma consulta — peça-chave da postura somente-leitura.

7.2. Engenheiro de Dados (text-to-SQL)

Escreve uma consulta SQL (SQLite) que cumpra o objetivo de cada passo. É o dono do loop de correção sintática: quando o SQL falha, ele recebe o erro real e regenera.

É o agente mais agêntico do sistema — vive no ciclo agir→observar→corrigir. Seu prompt embute as defesas aprendidas no estudo do banco: ancorar datas na âncora temporal, usar os valores categóricos exatos da tabela certa, e preferir a fonte transacional indicada nos alertas de qualidade.

Prompt de cargo (real, do código)

Placeholders entre parênteses e maiúsculas são preenchidos em tempo de execução (a pergunta, o dossiê, os resultados).

Você é um Engenheiro de Dados. Escreva UMA consulta SQL (SQLite) que cumpra o OBJETIVO DO PASSO abaixo, dentro do contexto da pergunta original do diretor.

REGRAS OBRIGATÓRIAS:

1. Apenas UM statement SELECT (WITH...SELECT permitido). Nada de INSERT/UPDATE/DELETE/DDDL/PRAGMA.
2. Datas relativas ancoram na ÂNCORA TEMPORAL do dossiê, nunca na data de hoje.
3. Use os valores EXATOS do dicionário de VALORES CATEGÓRICOS (grafia e capitalização idênticas, e da TABELA certa).
4. Respeite os ALERTAS DE QUALIDADE: prefira a fonte transacional indicada neles.
{bloco_erro}{bloco_auditor}{bloco_anteriores}
{dossie}

PERGUNTA ORIGINAL DO DIRETOR: {pergunta_original}

OBJETIVO DO PASSO: {objetivo_do_passo}

Contrato de saída (Pydantic)

```
class SqlDoPasso(BaseModel):
    sql: str # a consulta (apenas UM SELECT)
    justificativa: str # por que esta consulta cumpre o objetivo
```

Cada campo:

- `sql` — a consulta proposta; segue para o Guardrail antes de tocar o banco.
- `justificativa` — o raciocínio do Engenheiro; alimenta a transparência e ajuda o Auditor.

7.3. Auditor de Dados (autocorreção semântica, nível 2)

Avalia se os resultados respondem DE FATO à pergunta — não se o SQL é válido, mas se a resposta é a certa. Pode devolver ao Engenheiro com uma instrução de correção. “SQL válido com resposta errada é o pior erro possível.”

Existe para cumprir, com rigor, o requisito de autocorreção do enunciado no nível que separa um sistema pleno de um júnior: o nível semântico. Seu checklist codifica exatamente as conclusões do estudo do banco — fonte transacional, âncora temporal, domínios categóricos, empates no corte. A regra 5 reflete o desenho atual: o resultado completo já está com o sistema; o que aparece no prompt é só um recorte de exibição, e o Auditor não deve pedir “mais linhas”.

Prompt de cargo (real, do código)

Placeholders entre parênteses e maiúsculas são preenchidos em tempo de execução (a pergunta, o dossiê, os resultados).

Você é um Auditor de Dados. Avalie se os resultados abaixo respondem DE FATO a pergunta do diretor. SQL válido com resposta errada é o pior erro possível – seja rigoroso.

CHECKLIST OBRIGATÓRIO (verifique TODOS os pontos):

1. FONTE TRANSACIONAL: se o dossiê tem ALERTAS DE QUALIDADE sobre colunas divergentes, a consulta usou a fonte transacional recomendada (e NÃO a coluna denormalizada)?
2. ÂNCORA TEMPORAL: janelas relativas e meses sem ano foram ancorados na âncora temporal do dossiê (não na data de hoje)?
3. DOMÍNIOS CATEGÓRICOS: os filtros usam valores que EXISTEM na tabela consultada (mesma coluna pode ter domínios diferentes em tabelas diferentes)?
4. RESULTADO VAZIO: se vazio, é legítimo (não há mesmo dados) ou é sintoma de filtro/grafia errada? Confira contra o dicionário.
5. RECORTE DE EXIBIÇÃO: as linhas mostradas neste prompt são apenas um RECORTE para leitura – o resultado COMPLETO já está com o sistema e será usado integralmente no gráfico e na resposta. `n_linhas` informa o total real. NÃO devolva pedindo 'mais linhas' ou 'todas as linhas': elas JÁ estão completas.
6. EMPATES NO CORTE: em rankings top-N, verifique se há valores empatados na última posição que ficaram FORA do corte – se houver, a resposta deve revelar o empate, não escondê-lo.

Se reprovar: preencha `problema`, `instrucao_de_correcao` (clara e acionável) e `indice_passo_a_refazer` (0-based).

```
{dossie}
{bloco_dialogo}
PERGUNTA A AUDITAR (efetiva): {pergunta_a_auditar}
```

```
PREMISSAS DECLARADAS:
{premissas}
```

```
RESULTADOS A AUDITAR:
```

Contrato de saída (Pydantic)

```
class ParecerDoAuditor(BaseModel):
    aprovado: bool
    problema: str | None
    instrucao_de_correcao: str | None
    indice_passo_a_refazer: int | None # 0-based
```

Cada campo:

- `aprovado` — o veredito; a aresta lê este campo para seguir ao Visualizador ou devolver ao Engenheiro.
- `problema` — o que está errado, quando reprovado.
- `instrucao_de_correcao` — orientação acionável que entra no próximo prompt do Engenheiro.

- `indice_passo_a_refazer` — qual passo do plano regenerar (0-based).

7.4. Designer de Visualização

Autora o gráfico: produz o option do ECharts (JSON puro) que melhor comunica o resultado, com liberdade de escolher o tipo mais expressivo — barras, linha, pizza, dispersão, heatmap, Sankey, treemap, e mais.

Existe para cumprir a apresentação adaptativa com o máximo do potencial visual da biblioteca, em vez de um punhado de tipos fixos. Seu prompt proíbe explicitamente código executável (function, arrow, eval, JsCode, `<script>`) — a segurança da visualização está aqui e no guardrail, não na confiança. A regra 6z permite que um pedido explícito de tabela seja honrado pelo próprio Designer.

Prompt de cargo (real, do código)

Placeholders entre parênteses e maiúsculas são preenchidos em tempo de execução (a pergunta, o dossiê, os resultados).

Você é um Designer de Visualização de dados. Crie UM gráfico ECharts que melhor comunique o resultado abaixo a um diretor.

REGRAS OBRIGATÓRIAS:

1. `option_json` deve ser JSON PURO e completo (title, xAxis/yAxis quando aplicável, series com os dados EMBUTIDOS). É PROIBIDO qualquer código executável: function, arrow (`=>`), eval, new Function, JsCode, `<script>`, javascript:. 'formatter' só como template string (ex.: `'{b}:'` `{c}'`).
2. Apenas UM gráfico — jamais composições lado a lado.
3. Use SOMENTE os dados do RESULTADO abaixo — não invente valores.
4. Se a pergunta do usuário pediu um formato explícito (ex.: 'me mostre em pizza'), OBEDEÇA ao formato pedido.
5. Título e nomes de eixos em português, claros para leigos.
6. Escolha o tipo MAIS EXPRESSIVO e visualmente SINGULAR para a forma dos dados — VARIE entre: rosca/donut, radar, dispersão, heatmap, barras horizontais, área empilhada, funil, gauge, treemap, sunburst, linha multi-série, barras com gradiente... Fuja do óbvio quando outro tipo comunicar melhor. PROIBIDO map/geo (não suportados pelo renderizador).
- 6z. PEDIDO EXPLÍCITO DE TABELA: se a pergunta pede o resultado EM TABELA (ex.: 'mostre em tabela', 'formato de tabela'), responda com `tipo_grafico='tabela'` e `option_json={}` (vazio) — a interface renderiza a tabela nativa com os dados. NÃO force um gráfico nesse caso.
- 6c. GALERIA PREFERENCIAL: quando a pergunta fizer SENTIDO MATEMÁTICO para um destes, PRIORIZE-O (todos em JSON puro do ECharts):
 - Variação ao longo do tempo (estilo 'Temperature Change in the Coming Week'): line multi-série com `markPoint/markLine`;
 - Punch card (intensidade em 2 dimensões discretas): scatter cartesiano com `symbolSize` proporcional (o bar3D clássico exige echarts-gl, INDISPONÍVEL no runtime — use esta forma);
 - Pie with `padAngle`: pie com `padAngle` e `itemStyle.borderRadius`;
 - Scatter Polynomial Regression (relação entre 2 métricas numéricas): scatter; se houver até ~60 pontos, adicione uma série line suave (`smooth`) com os pontos ordenados pelo eixo X como tendência aproximada (o plugin ecStat NÃO existe no runtime);
 - Heatmap on Cartesian: heatmap com `visualMap`;
 - Tree From Left to Right / Right to Left: tree com orient 'LR' ou 'RL' (hierarquias);
 - Treemap (partes de um todo com pesos); Sunburst (hierarquia radial); Sankey (fluxos origem→destino); Funnel (etapas que afinam); Gauge (um indicador contra meta); PictorialBar (contagens com `symbolRepeat`);
 - Calendar Graph: coordenada calendar com heatmap/scatter (valores por dia);
 - Matrix: matriz densa como heatmap cartesiano;

- Chord (relações entre categorias): series graph com layout 'circular' (o chord clássico não existe no ECharts 5).

Se NENHUM da galeria fizer sentido matemático para a pergunta, escolha LIVREMENTE qualquer tipo da biblioteca que comunique melhor. PROIBIDO: map/geo, bar3D/scatter3D (exigem extensões ausentes) e qualquer código executável.

6b. CAPRICHE NA ESTÉTICA: paleta harmoniosa no option (campo 'color'), borderRadius nas barras, rótulos legíveis, tooltip configurado.

7. ARREDONDE valores numéricos exibidos nos dados do option para 2 casas decimais – nunca embute floats crus (ex.: 1.8823529...).

```
{bloco_reprova}
```

```
PERGUNTA DO USUÁRIO: {pergunta}
```

```
OBJETIVO RESPONDIDO: {justificativa_da_resposta}
```

```
RESULTADO (dados a visualizar):
```

```
Colunas: {resultado['colunas']}
```

```
Linhas ({resultado['n_linhas']}): {resultado['linhas']}
```

Contrato de saída (Pydantic)

```
class PropostaDoDesigner(BaseModel):
    tipo_grafico: str      # barra, linha, pizza, dispersao, heatmap... (informativo)
    option_json: str       # o option ECharts COMPLETO, JSON PURO, dados embutidos
    justificativa: str     # por que este gráfico serve a esta pergunta
```

Cada campo:

- `tipo_grafico` — o tipo proposto; informativo, ajuda o log e o fallback.
- `option_json` — o option completo como string JSON pura, com os dados embutidos; proibido qualquer código executável.
- `justificativa` — 1–2 frases sobre por que o gráfico serve à pergunta.

7.5. O Diretor (segunda função do Analista)

A redação final é feita pelo mesmo LLM sob um cargo de redação: transforma os resultados em uma resposta de negócio, sem jargão, com as premissas em destaque e uma conclusão de impactos e ações. Na exibição da interface, esse papel aparece como “Diretor”; o identificador interno do nó permanece estável.

Prompt de cargo (real, do código)

Você é um Analista de Negócios redigindo a resposta final para um diretor (linguagem clara, sem jargão técnico).

REGRAS OBRIGATÓRIAS:

1. Use APENAS os números e fatos dos RESULTADOS abaixo. É PROIBIDO inventar, extrapolar ou 'completar' valores.
2. Se os resultados estiverem vazios, diga honestamente que não há dados para responder – não invente.
3. Liste em `premissas_destacadas` TODAS as premissas assumidas.
4. Dados truncados: deixe claro que a leitura é sobre uma amostra.
5. CONCLUSÃO EXECUTIVA (campo `impactos_e_acoes`): escreva EXATAMENTE 2 parágrafos, fundamentados NOS DADOS desta resposta. O parágrafo 1 COMEÇA com o prefixo 'IMPACTOS PARA O

NEGÓCIO:' (em maiúsculas) seguido do texto em CAPITALIZAÇÃO NORMAL (não use CAIXA ALTA no corpo) sobre os impactos que esses números revelam (riscos, oportunidades, o que está em jogo). O parágrafo 2 COMEÇA com o prefixo 'AÇÕES:' (em maiúsculas) seguido do texto em capitalização normal sobre as ações concretas para amenizar consequências negativas e impulsionar os resultados. Em CADA parágrafo, destaque as frases mais importantes envolvendo-as em **negrito markdown** (ex.: **risco de dependência**), mesmo em minúsculas. Linguagem de diretor, acionável, sem jargão. Se os resultados forem insuficientes, diga-o com honestidade (não invente impacto sem dado).

6. VALORES FORA DO PADRÃO: se o bloco abaixo apontar valores com $|z| > 2$ (outliers detectados deterministicamente), comente-os na conclusão executiva – são o que mais merece a atenção do diretor (uma concentração, um desvio, uma exceção). Não invente outliers além dos listados.

PERGUNTA ORIGINAL: {pergunta}

PREMISSAS DA ANÁLISE:

{premissas}

{bloco_outliers}

RESULTADOS:

Contrato de saída (Pydantic)

```
class RespostaDeNegocio(BaseModel):
    resposta: str # linguagem de diretor, sem jargão
    premissas_destacadas: list[str] # o que foi assumido, em destaque
    impactos_e_acoes: list[str] # a conclusão executiva
```

Cada campo:

- `resposta` — o texto final em linguagem clara para um decisor.
- `premissas_destacadas` — as premissas assumidas, listadas para transparência.
- `impactos_e_acoes` — a conclusão executiva: o que o número significa para o negócio e o que fazer a respeito.

8. Os componentes determinísticos (Python, não-LLM)

São peças de Python puro, sem LLM. **Não são agentes**: dão sempre a mesma saída para a mesma entrada, e é exatamente por isso que existem. A metáfora do organograma fica completa — agência onde há incerteza, determinismo onde há mecânica.

8.1. Perfilador do Banco

Constrói e cacheia o dossiê de schema enriquecido (as 6 camadas — seção 10). É determinístico porque descobrir a estrutura é mecanismo, não juízo: o mesmo banco produz sempre o mesmo dossiê, testável byte a byte.

8.2. Guardrail SQL

Decide se uma consulta pode tocar o banco: é só leitura? É um único statement? Não há UPDATE/INSERT/DELETE/DROP, PRAGMA, ATTACH? É determinístico porque é a fronteira de segurança — precisa ser a mesma decisão, sempre, auditável e imune a persuasão. Um guardrail-LLM seria um vigia probabilístico: 0,1% de “neste caso é razoável” é uma porta aberta, inclusive à injeção de prompt.

Quem valida não pode ser da mesma natureza de quem é validado — senão herdaria a mesma classe de falhas que deveria conter.

8.3. Guardrail de Visualização (interno ao nó visualizador)

Valida o option proposto pelo Designer: varre recursivamente chaves e valores em busca de código executável (function, =>, eval, new Function, JsCode, <script>, javascript:), exige a presença de series e impõe limites de tamanho e profundidade. Aceitar JS gerado por LLM seria abrir porta de injeção de código no navegador. Ele é representado no diagrama como uma anotação tracejada ligada ao nó, não como um nó próprio — porque vive **dentro** do laço Designer→validação.

8.4. Executor Somente-Leitura

Abre a conexão SQLite em modo `mode=ro` na URI, executa o SQL aprovado e cronometra. É determinístico porque é mecânica pura — não há decisão semântica que uma inteligência resolveria melhor. A conexão read-only é a garantia **física**: ainda que algo escapasse do guardrail, o banco recusaria a escrita. É defesa em profundidade.

O mapa da confiança: as arestas de autocorreção (mostarda) só existem entre agentes; os determinísticos nunca precisam ser corrigidos, porque nunca improvisam.

9. A lógica dos fluxos e os contratos em ação

As arestas condicionais leem campos específicos dos contratos para rotear. O quadro a seguir mostra quem lê o quê, e o teto que contém cada laço:

Campo lido pela aresta	Decisão de roteamento	Teto (determinístico)
AnaliseDaPergunta.precisa_esclarecimento	esclarecer (interrupt) ou engenheiro	MAX_ESCLARECIMENTOS = 3 → falha graciosa
AnaliseDaPergunta.pedido_fora_de_escopo	recusa honesta direta (somente leitura)	—
SqlDoPasso.sql	Guardrail aprova → executor; reprova → engenheiro	MAX_TENTATIVAS_SQL = 3
ParecerDoAuditor.aprovado	visualizador ou devolução ao engenheiro	MAX_DEVOLUCOES_AUDITOR = 15

Campo lido pela aresta	Decisão de roteamento	Teto (determinístico)
PropostaDoDesigner.option_json	Guardrail de visualização; reprova → retry → tabela	MAX_TENTATIVAS_VISUAL = 2

Acima de todos os tetos locais há o **orçamento global**: `ORCAMENTO_GLOBAL_CHAMADAS_LLM = 40` chamadas por pergunta. A regra de coerência (testada) garante que o orçamento global comporta o teto do Auditor — 15 devoluções, a ~2 chamadas cada, mais o plano base e o visual.

Um exemplo concreto

Pergunta: “qual a tendência de reclamações por canal no último ano?”. O Analista ancora “último ano” em 2025-07-22 (premissa declarada) e monta um passo. O Engenheiro escreve o SQL; o Guardrail aprova; o Executor roda. O Auditor verifica a âncora e os domínios de `canal` em suporte e, num caso real de fogo, devolveu uma vez exigindo a grade completa `mês×canal` — o Engenheiro regenerou com uma CTE recursiva. Aprovado, o Designer autora uma linha multi-série; o Diretor redige a resposta avisando que julho/2025 é parcial. Todo esse caminho fica registrado no trace e desenhado no diagrama da mensagem.

10. O dossiê do banco (schema enriquecido, gerado dinamicamente)

O Perfilador não recebe nada escrito por nós: ele **descobre tudo sozinho, em runtime, contra qualquer SQLite**. “Schema enriquecido” é o schema cru (que todo mundo fornece) somado a cinco camadas de contexto que quase ninguém fornece. O resultado, renderizado como texto por `renderizar_para_prompt`, é injetado no prompt de **todos** os agentes e calculado uma vez por processo (cache). O título da saída é literalmente `=== DOSSIÊ DO BANCO DE DADOS (gerado dinamicamente) ===`.

As 6 camadas

Camada	O que entrega	O que ela resolve
1. Estrutura	Tabelas, colunas, tipos, PK e FK (via PRAGMA)	O Engenheiro só escreve SQL sobre o que existe
2. Volumetria	Contagem de linhas por tabela	Régua de sanidade: um COUNT implausível denuncia JOIN explodido
3. Amostras	Até N linhas reais por tabela	Mostra formatos reais (datas ISO, estado por extenso)
4. Dicionário categórico	Todos os valores das colunas de baixa cardinalidade	Os domínios disjuntos de canal: filtrar na tabela certa, com a grafia exata
5. Perfil temporal	Mín/máx e formato de cada coluna de data; gera a ÂNCORA	Janelas relativas ancoram no MAX(data), não em hoje
6. Qualidade	Detector genérico de denormalização, com % e recomendação	As colunas redundantes desatualizadas: prefira a fonte transacional

O Perfilador é um termômetro: dinâmico sem ser inteligente

A camada 6 não é uma regra decorada para este banco. É um **mecanismo**: o detector percorre as FKs reais (via `PRAGMA foreign_key_list`) e compara a coluna do pai com o agregado da filha (total/soma → SUM, última/máx → MAX), medindo a fração divergente. Funciona em qualquer banco com colunas denormalizadas — e uma varredura dos fontes prova que os nomes das colunas-armadilha deste banco não existem no código.

O Perfilador é dinâmico sem ser inteligente: adapta-se ao banco por um mecanismo, não por um juízo. Adaptar-se à estrutura é descoberta; interpretar a pergunta é agência.

Determinismo não é rigidez — é a garantia de que a mesma realidade sempre produz o mesmo mapa.

Trecho real dos alertas (banco do desafio)

A saída real da camada 6 contra o banco do desafio inclui:

```
ALERTAS DE QUALIDADE DOS DADOS
clientes.valor_total_gasto diverge de SUM(compras.valor)
```

```
em 100% das linhas – prefira a fonte transacional (compras).
clientes.data_ultima_compra diverge de MAX(compras.data_compra)
em 100% das linhas – prefira a fonte transacional (compras).
```

É por isso que nenhuma consulta da bateria do enunciado tocou as colunas-atalho: o agente nasce vacinado, e o Auditor tem a evidência para conferir.

11. Escalabilidade do dossiê

O desenho atual tem premissas que só valem em banco pequeno. A resposta honesta começa pelo que quebra.

O que quebra num banco gigante

- **O dossiê monolítico não cabe no prompt.** 5.000 caracteres para 4 tabelas é grátis; um ERP com 2.000–10.000 tabelas geraria milhões de tokens — não cabe, custa caro e piora a precisão (o LLM se perde em contexto irrelevante).
- **O perfilamento em runtime vira ataque ao banco.** COUNT, DISTINCT por coluna e as agregações da camada 6 são full scans — horas em tabelas de bilhões de linhas, competindo com a produção.
- **O cache por processo não sobrevive.** Banco gigante muda o tempo todo; “calculei no startup” vira dossiê mentiroso.

Os três movimentos da migração

Movimento 1 — perfilamento sai do caminho da pergunta. Vira job offline (noturno/por migração de schema), persistido num catálogo; usa as estatísticas que o banco já mantém de graça (pg_stats, information_schema), TABLESAMPLE para amostras, e reperfilamento incremental (só o que mudou).

Movimento 2 — o dossiê vira recuperável (schema linking / RAG). Em vez de entregar tudo a todos, um recuperador seleciona as top-k tabelas relevantes por pergunta (busca lexical + semântica) e expande pela vizinhança de FKs, montando um mini-dossiê. E aqui está o ponto arquitetural central: **nada nos agentes muda** — o contrato é estado["dossie"]; só a construção troca de “tudo” para “o recorte relevante”.

Movimento 3 — fusão com a governança. Em escala corporativa, dicionário e alertas vivem no catálogo de dados (DataHub, Unity Catalog, dbt docs) e nas suítes de qualidade (dbt tests, Great Expectations). O nosso Perfilador vira o bootstrap que gera a primeira versão do catálogo.

O dossiê é um contrato; em banco pequeno é computado inteiro em runtime, em banco gigante é um catálogo perfilado offline do qual um recuperador extrai o recorte por pergunta — os agentes nem ficam sabendo da diferença.

A execução também ganharia cintos extras: réplica de leitura (jamais o primário), timeout de consulta e um guardrail de custo com EXPLAIN, rejeitando planos de full scan acima de um limiar. A separação determinístico×LLM é exatamente o que torna a migração barata.

12. Três cenários de arquitetura: o piso, o teto e o ponto ótimo

Para situar a decisão, comparo três arquiteturas que formam um gradiente de agência: pouca, demais e calibrada. Não são os únicos cenários possíveis; são os três que melhor demonstram por que a dosagem importa.

Critério	A — Chain fixa	E — Multi-agente puro	D — Grafo estruturado (nosso)
Agência	Nenhuma	Total (supervisor-LLM)	Calibrada por nó
Autocorreção	Não há	Possível, sem teto natural	Sintática + semântica, com tetos
Segurança	Confia no prompt	Vulnerável a injeção entre agentes	Guardrail determinístico + read-only
Descoberta de schema	Ausente / hardcoded	Depende do supervisor	Perfilador determinístico
Não-determinismo	Baixo, mas frágil	Alto (rota imprevisível)	Baixo (rota fixa, agência nos passos)
Custo de loop	—	Orçamentos por agente E global	Tetos locais + orçamento global

A — Chain fixa (o piso)

Pergunta → gera SQL → executa → responde. Sem loop, sem validação, sem guardrail. É o que muita gente entrega — e por isso é o espantalho honesto. **O que falta:** um SQL errado morre ali (sem autocorreção); as colunas-armadilha do banco passariam despercebidas (sem descoberta nem auditoria); e um `DROP TABLE` embutido na pergunta tocaria o banco, porque **a única defesa seria confiar no prompt** — e a segurança não pode morar no prompt.

E — Multi-agente puro com supervisor-LLM (o teto, o excesso)

Um LLM-supervisor decide dinamicamente qual agente chamar a seguir, sem roteamento fixo. É agência demais para um problema cujo fluxo macro já conhecemos. Além do não-determinismo na rota, ele **adiciona riscos que o nosso desenho não tem:**

O E puro adiciona ainda um risco que o D não tem: injeção entre agentes (um resultado de query contendo texto malicioso que vira instrução para o próximo agente) e necessidade de orçamentos de loop por agente e globais. Mais um motivo para o roteamento determinístico: ele limita estruturalmente o que uma injeção consegue desviar.

D — Grafo estruturado custom (o nosso, o ponto ótimo)

Agência concedida onde há incerteza (planejar, escrever SQL, julgar, desenhar), determinismo onde ela é perigosa (roteamento por arestas condicionais, guardrails, execução, tetos). O fluxo macro é fixo e conhecido; a incerteza vive **dentro** dos passos, não entre eles. É a síntese entre o piso e o teto: resolve o que o A não resolve, sem incorrer nos riscos que o E cria.

13. Por que arestas condicionais, e não um supervisor-LLM

O roteamento entre os agentes é por arestas condicionais, não por supervisor-LLM, porque o fluxo macro deste problema é conhecido (planejar → consultar → analisar → visualizar; a incerteza vive dentro dos passos, não entre eles) — colocar um LLM para decidir um roteamento que já sabemos seria adicionar não-determinismo onde ele não compra nada. Na entrevista, isso se apresenta legitimamente como arquitetura multiagente com a justificativa de maturidade: agência onde há incerteza, determinismo onde há mecânica.

Há ainda o dividendo de segurança visto na seção 12: um roteamento determinístico limita estruturalmente o que uma injeção entre agentes conseguiria desviar — o caminho não muda porque um texto malicioso “pediu” para mudar.

14. Segurança

A segurança é em camadas, e toda ela é determinística — porque a fronteira de segurança não pode ser probabilística.

Defesa em profundidade

1. **Guardrail SQL.** Aceita apenas um único SELECT (WITH...SELECT permitido); rejeita INSERT/UPDATE/DELETE/DROP, PRAGMA, ATTACH e multistatement; remove comentários antes da análise (testado contra evasões como SELECT 1 /* */; DROP).
2. **Conexão somente-leitura.** O Executor abre o SQLite com `mode=ro` — a garantia física: testes de escrita que chamam o Executor sem passar pelo guardrail são recusados pelo próprio banco, com contagem independente confirmando-o intacto.
3. **Guardrail de Visualização.** O option do gráfico é varrido contra qualquer código executável (function, arrow, eval, JsCode, <script>); só template strings de formatter passam. Aceitar JS gerado por LLM seria injeção de código no navegador.
4. **Recusa de escrita na origem.** O Analista marca `pedido_de_escrita / pedido_fora_de_escopo`; a aresta roteia direto para uma recusa honesta, sem nem chegar ao Engenheiro.

O LLM propõe, o determinismo dispõe — quem pensa pode errar, então quem protege não pode pensar. A segurança mora na ferramenta, nunca no prompt do agente.

Há também uma razão lógica: **quem valida não pode ser da mesma natureza de quem é validado**. Se o guardrail fosse outro LLM, herdaria a mesma classe de falhas do Engenheiro que deveria conter — empilharia incerteza em vez de contê-la.

15. Visualização: dois autores, um renderizador

A visualização tem dois modos possíveis de autoria, mas um único renderizador (o ECharts), e um único gráfico por resposta — jamais composições lado a lado.

Modo agêntico (o escolhido como padrão)

O Designer autora o option completo do ECharts conforme a pergunta pede. Isso libera todo o leque da biblioteca — pizza, dispersão, heatmap, Sankey, treemap, multi-série — em vez de um punhado de tipos fixos. **Foi a escolha deliberada para demonstrar profundidade técnica**, dado o alto interesse nesta vaga. O fallback fica trivial: option inválido após as tentativas → tabela. A heurística vira uma linha, não uma taxonomia.

Modo pre_setado (a alternativa determinística)

Poderíamos ter usado apenas uma heurística que respeita a natureza do dado:

- 1 valor → métrica;
- lista → tabela;
- série temporal → linha;
- comparação categórica → barra.

É tecnicamente mais simples e 100% previsível, mas comunica menos. Por isso é a alternativa, não o padrão. **Recomendação ao avaliador:** use o **modo Agente** (selecionável na barra lateral) — foi o exaustivamente testado nos fogos. O modo pre_setado, além de visualmente inferior, não passou por uma bateria de testes equivalente.

Outliers em destaque

Quando uma tabela é exibida, um detector determinístico de outliers ($z\text{-score} > 2$) marca as linhas fora do padrão com destaque visual e uma legenda. O mesmo detector alimenta a conclusão do Diretor — **análise e apresentação ligadas pela mesma fonte de verdade**, sem duplicar regra.

O limite honesto do ECharts

Seremos sempre sinceros sobre os limites: o guardrail valida *estrutura e segurança* do JSON, **não a renderabilidade** de toda combinação do ECharts. Tipos que exigem extensões externas (mapas geográficos, 3D) não são suportados e caem para tabela. Para explorar a variedade possível, vale a galeria oficial: [Apache ECharts — Examples](#).

16. Avaliação de modelos de mercado

Em uma implementação profissional, a escolha do modelo de cada agente é uma decisão de custo×performance — e um sistema multiagêntico pode ser também **multiplataforma**, usando o modelo certo em cada papel. A tabela abaixo reúne preços públicos por milhão de tokens (verificados em junho de 2026; valores mudam com frequência):

Modelo (provedor)	Entrada (US\$/1M)	Saída (US\$/1M)	Observação
Gemini 2.5 Flash (Google)	0,30	2,50	Usado aqui na classificação/decisões
Gemini 2.5 Pro (Google)	1,25	10,00	Usado aqui no raciocínio pesado

Modelo (provedor)	Entrada (US\$/1M)	Saída (US\$/1M)	Observação
DeepSeek V4 Flash	0,14	0,28	O mais barato; forte custo-benefício
Claude Sonnet 4.6 (Anthropic)	3,00	15,00	Frontier; forte em código/agêntico
GPT-5.4 (OpenAI)	2,50	15,00	Frontier; generalista
Perplexity Sonar Pro	3,00	15,00	Busca web embutida com citações

Preços públicos por 1M de tokens, padrão ($\leq 200K$), coletados em junho de 2026; sujeitos a alteração pelos provedores.

Por que Gemini neste projeto

Foi usado o Gemini porque é onde tenho conta e chave configuradas — uma decisão de contexto, não de superioridade absoluta. Em produção, a arquitetura permitiria **misturar provedores por papel**: um modelo barato (Flash-Lite, DeepSeek) para classificação e decisões simples, um modelo forte (Pro, Sonnet) só onde a qualidade do SQL ou do julgamento paga o custo. O par `temperature=0` e constantes centrais em `config.py` mantém isso trocável em um ponto só.

Benchmark do nosso Engenheiro

O Engenheiro de Dados — quem monta a query — teve comportamento excelente nos fogos. Ainda assim, o passo profissional seguinte é **comparar formalmente** o desempenho dele contra ferramentas de mercado (LangChain SQL Agent, PandasAI, um `text2sql` do Hugging Face) sobre uma bateria fixa com gabarito determinístico — para certificar que o desempenho não poderia ser ainda melhor com uma ferramenta especializada. É o trabalho mapeado para um marco de benchmark.

17. Escalabilidade, custo, robustez, latência e testabilidade

Para cada atributo, o que já foi aplicado e o que viria nas próximas versões.

Escalabilidade

- **Aplicado:** separação determinístico×LLM que torna a migração barata; dossiê com contrato estável.
- **Próximas versões:** perfilamento offline + recuperação por pergunta (seção 11); execução contra réplica de leitura.

Custo

- **Aplicado:** constantes centrais; orçamento global de chamadas por pergunta; cache do dossiê por processo.
- **Próximas versões:** seleção multiplataforma de modelos por papel; context caching; modelo barato na classificação.

Robustez

- **Aplicado:** três laços de autocorreção com tetos; detector de ciclo improdutivo (corta SQL repetido pós-devolução); degradação graciosa do gráfico para tabela; falha graciosa explicativa em todo teto.
- **Próximas versões:** retry com backoff em erros transitórios de API; circuit breaker por provedor.

Latência interativa

- **Aplicado:** streaming do raciocínio nó-a-nó (o usuário vê o progresso ao vivo, não espera em silêncio); o diagrama acende conforme os eventos chegam.
- **Próximas versões:** paralelização de passos independentes do plano; modelos mais rápidos nos papéis de baixa criticidade.

Testabilidade e depurabilidade

- **Aplicado:** componentes determinísticos testáveis byte a byte; motor headless separado da UI (testável sem navegador); trace estruturado e observabilidade no LangSmith; relatório de testes legível.
- **Próximas versões:** replay de sessões a partir do trace persistido; métricas de acerto por tipo de pergunta.

18. Metodologia de desenvolvimento

O projeto foi conduzido por quatro princípios que se reforçam, num ciclo de vida incremental dividido em marcos.

18.1. SDD — Spec-Driven Development

Cada marco começou por uma SPEC revisada e aprovada antes de qualquer código. A spec contém as assinaturas das funções, a tabela de microatividades e a tabela de testes (nome e o que cada um testa). Nenhuma linha foi escrita sobre suposição: decisões de comportamento foram fechadas no papel primeiro.

18.2. TDD — Test-Driven Development orientado a risco

O objetivo final foi decomposto em microatividades testáveis. Cada microatividade tem mais de um teste, priorizando os pontos vulneráveis (segurança, entradas inválidas, casos de borda, invariantes) em vez do caminho feliz. O teste mais crítico do projeto é o do guardrail contra SQL malicioso.

18.3. Orientação a análise de critério

Cada escolha foi justificada por critérios explícitos — não “fiz assim”, mas “escolhi A em vez de B porque...”. Este documento é, em boa medida, o registro desses critérios.

18.4. Orientação a revisão

Todo entregável passou por revisão antes de avançar, e a documentação é evolutiva: cada marco tem um registro complementar realimentado com os resultados reais, inclusive a saída dos testes de fogo.

18.5. Os cinco marcos

O ciclo respeitou o princípio “motor antes de interface”:

5. **Marco 1 — Fundação + Perfilador.** Estrutura do projeto, config (modelos, temperatura=0) e os determinísticos da base: Perfilador (6 camadas, incluindo o detector genérico de denormalização), Guardrail SQL e Executor. Fecha com a prova mínima de fogo (1 pergunta → 1 SQL → resposta).
6. **Marco 2 — O grafo multiagêntico.** LangGraph com os agentes e o roteamento determinístico; Analista (decomposição, premissas, interrupt), Engenheiro (loop sintático), Auditor (nível 2 semântico); planos multi-passo. Saída em texto + trace, sem UI.
7. **Marco 3 — Visualização.** Dois autores, um renderizador; Designer autorando ECharts; Guardrail de Visualização (JSON puro); fallback tabela. Um gráfico por resposta.
8. **Marco 4 — Interface Streamlit + transparência.** Chat com histórico, streaming do raciocínio por nó, dois níveis (resposta + trace técnico), o interrupt aflorando na tela, observabilidade ativa.

9. **Marco 5 — Visualização dinâmica do grafo + endurecimento.** O diagrama do caminho por mensagem (ao vivo e replay), a identidade FRANQ, e correções críticas vindas do fogo (o anti-loop do esclarecimento, o detector de ciclo improdutivo, o destaque de outliers).

Um marco de benchmark comparativo (contra LangChain SQL Agent, PandasAI e text2sql) está mapeado como trabalho seguinte.

19. Organização sistêmica do código

19.1. Tabela dos módulos e suas interações

Módulo (app/)	O que faz e como interage
perfilador.py	DETERMINÍSTICO. Constrói o dossiê (6 camadas) do SQLite; o nó perfilador o injeta no estado para todos os agentes.
detector_denormalizacao.py	DETERMINÍSTICO. A camada 6: compara coluna do pai vs agregado da filha via FK; usado pelo Perfilador.
guardrail_sql.py	DETERMINÍSTICO. Valida o SQL do Engenheiro antes de tocar o banco; a aresta lê o veredito.
executor.py	DETERMINÍSTICO. Abre conexão read-only e executa o SQL aprovado; devolve linhas ao estado.
analise_outliers.py	DETERMINÍSTICO. Detector de outliers (z-score); alimenta o Diretor e o destaque da tabela.
agentes/analista_negocios.py	AGENTE. Analisa a pergunta (plano/premissas) e redige a resposta final (Diretor).
agentes/engenheiro_dados.py	AGENTE. Escreve o SQL de cada passo; consome erros e instruções de correção.
agentes/auditor_dados.py	AGENTE. Julga se o resultado responde à pergunta; pode devolver ao Engenheiro.
agentes/designer_visualizacao.py	AGENTE. Autora o option ECharts; validado pelo guardrail de visualização.
estado.py	O estado do grafo (TypedDict) e os contratos Pydantic; o vocabulário comum de todos os nós.
grafo.py	Monta o grafo: registra os nós e as arestas condicionais; é a topologia.
fluxo.py	Motor de streaming headless; traduz o grafo em eventos (trace/interrupt/final). A UI não conhece LangGraph.
config.py	Constantes centrais: modelos, temperatura=0, tetos, nome do projeto LangSmith.
trace.py / cores.py	O registro do raciocínio por nó e os nomes/cores de exibição por papel.
interface.py	A UI Streamlit: chat, trace ao vivo, gráfico, diagrama do caminho, SQL na tela.

Módulo (app/)	O que faz e como interage
principal.py / bateria.py	CLIs: pergunta avulsa e as perguntas do enunciado de uma vez.
visualizacao/grafico_svg.py	O diagrama do caminho percorrido (SVG estático, sem script).
visualizacao/guardrail_visualizacao.py	DETERMINÍSTICO. Varre o option contra código executável; interno ao nó visualizador.
visualizacao/templates.py / formas.py	Modo pre_setado: classificador da forma do dado + templates determinísticos.
visualizacao/render_streamlit.py	Decisão pura de renderização (echarts/métrica/tabela/nada), testável sem Streamlit.

19.2. Assinaturas-chave por arquivo

perfilador.py

```
def perfilar(caminho_banco: Path) -> PerfilDoBanco      # cacheado por processo
def renderizar_para_prompt(perfil: PerfilDoBanco) -> str # o dossiê como texto
```

guardrail_sql.py

```
def validar(sql) -> ResultadoValidacao # aprovado? + motivo da reprova
```

executor.py

```
def executar(sql, caminho_banco) -> ResultadoDaExecucao # conexão mode=ro
```

fluxo.py

```
def criar_sessao(...) -> Sessao          # mesmo grafo/checkpointer na sessão
def executar(sessao, pergunta) -> Iterator[Evento] # trace | interrupt | final
def retomar(sessao, resposta) -> Iterator[Evento]  # pós-interrupt
```

19.3. A lógica da estrutura de pastas

Os módulos na raiz de app/ são as peças do motor; agentes/ isola os papéis de LLM (cada um com prompt e contrato); visualizacao/ agrupa autoria, validação e renderização de gráficos. **A separação física espelha a tese**: determinísticos e agentes vivem em camadas distintas e nomeadas, e a UI (interface.py) fala com o motor por eventos, nunca com o LangGraph direto — o que mantém o motor testável e headless.

20. Os testes

A suíte do projeto final tem **232 testes**, orientados a risco. A base lógica: para cada microatividade, mais de um teste, priorizando o que ameaça o objetivo — segurança, entradas inválidas, bordas, invariantes — em vez do caminho feliz. Abaixo, uma amostra representativa do que cada teste trava:

Teste (amostra)	O risco que ele trava
guardrail contra SQL malicioso	DROP/UPDATE/multistatement/PRAGMA barrados; o teste mais crítico
executor sem passar pelo guardrail	Defesa em profundidade: o mode=ro recusa escrita fisicamente
anti-overfitting por varredura dos fontes	As colunas-armadilha não existem no código; o detector as acha sozinho
loop sintático (mock erra K vezes)	Converge na 3ª? Respeita o teto? Falha graciosamente errando sempre?
devolução do Auditor com instrução	O passo é regenerado com a instrução; o teto de devoluções é respeitado
detector de ciclo improdutivo	SQL repetido pós-devolução corta o ciclo na hora (falha graciosa específica)
interrupt de ambiguidade	Pausa, a resposta vira premissa, o fluxo conclui; teto de esclarecimentos corta o loop
orçamento global	Roteiro patológico cortado em ≤40 chamadas com falha graciosa
injeção de JS no option (13 casos)	100% barrada; formatter como template passa
coerência topologia × grafo real	O diagrama nunca diverge silenciosamente do grafo compilado
tabela destaca outlier (z>2)	A linha fora do padrão é marcada; sem outlier, sem falso positivo
resultado vazio legítimo	Responde “não há dados” — o Diretor não inventa

O que os testes garantiram

Mais do que cobertura, os testes documentam um ciclo de engenharia real: o fogo do Marco 2 revelou uma falha de prompt no Auditor (um diagnóstico errado de truncamento), a causa foi identificada com evidência, e a correção entrou com regressão — o checklist evoluiu de 5 para 6 pontos. **Validação empírica e documentação evolutiva em ato, não em discurso.**

Os testes que dependem de chamadas reais ao Gemini são marcados e exercitados nos testes de fogo (manuais); a suíte determinística roda sem rede, com o LLM mockado — por isso é reproduzível em qualquer máquina.

21. Sobre o LangGraph Studio: dossiê e diagnóstico

O `langgraph.json` criado é, por si só, o artefato de deploy: é exatamente o manifesto que o LangGraph Platform usa para publicar um grafo em produção. Ele registra o grafo assistente apontando para `app/studio.py:grafo`.

O que foi comprovado

- **O log do servidor:** o Agent Server subiu registrando `graph_id=assistente`, `langgraph-api=0.10.0`, com metadados enviados ao LangSmith.
- **A API respondendo:** a rota `/assistants/search` devolveu um JSON válido (rejeitando um ID malformatado com “must be a UUID”) — ou seja, o servidor está vivo e é o nosso; o túnel HTTPS funciona; o deploy está comprovado de ponta a ponta.
- **O manifesto:** `langgraph.json` + `studio.py` são a peça que importa para publicação.

O diagnóstico honesto da falha visual

A interface web do Studio não renderizou no navegador (a página `/docs` ficou em branco mesmo na versão estável, e o seletor caía no template genérico de 4 nós). Como o servidor comprovadamente responde, a conclusão é que **o problema é a renderização das páginas de terceiros no navegador, não o nosso código (perfeito nos testes), não o servidor, não o deploy**.

A causa-raiz foi um ciclo que não fecha — túnel expira → reinicia → “mixed content” → domínio não autorizado → autoriza → “failed to fetch” → o túnel gratuito expira de novo antes de a conexão completar. **Isso não é um bug a corrigir; é uma limitação fundamental da infraestrutura efêmera gratuita dos túneis** (trycloudflare.com), que o próprio LangGraph adverte não ter garantia de uptime. O Studio é opcional; a observabilidade no LangSmith (projeto `d1_franq`) já funciona com o `streamlit run`. A substância do deploy está provada — a tela bonita seria a cereja, não o bolo.

22. Heurísticas de fallback (degradação graciosa)

O sistema nunca quebra na cara do usuário; ele degrada em camadas, sempre devolvendo o melhor possível dado o que falhou:

- **Gráfico → tabela:** se o option não validar após as tentativas, renderiza a tabela (porto seguro universal). A visualização jamais derruba a resposta.
- **SQL → nova tentativa → falha graciosa:** erro de execução realimenta o Engenheiro; esgotado o teto, uma mensagem honesta explica o último erro.
- **Ambiguidade → esclarecimento → recomeço:** até 3 perguntas ao usuário; na 4ª, a falha graciosa pede reformulação em vez de insistir.
- **Ciclo improdutivo → corte imediato:** se a correção repete o SQL reprovado, a aresta corta na hora e sugere pedir um agregado ou aceitar uma amostra.

- **Resultado vazio legítimo:** o Diretor recebe zero linhas e responde “não há dados” — não inventa.

23. Limitações

De infraestrutura e escala

- O dossiê é computado inteiro em runtime — ótimo para dezenas de tabelas, inadequado para milhares (o plano de escala está na seção 11).
- O Studio depende de túnel efêmero gratuito sem garantia de uptime (seção 21).

Da lógica dos agentes

- A variabilidade residual do LLM existe mesmo a temperatura 0; é contida por contratos, guardrails e tetos, não eliminada.
- O roteamento macro é fixo por decisão — o sistema não reorganiza o fluxo diante de uma pergunta de natureza inteiramente nova (é uma escolha de engenharia, seções 12–13).

Dos componentes determinísticos

- O detector da camada 6 parecia colunas por afinidade de nomes; uma coluna denormalizada com nome opaco escaparia (limite documentado do mecanismo).
- O guardrail de visualização valida estrutura e segurança, não a renderabilidade de toda combinação do ECharts (tipos com extensões externas caem para tabela).

24. Sugestões de melhorias e próximas versões

10. **Benchmark comparativo** do Engenheiro de SQL contra LangChain SQL Agent, PandasAI e um text2sql do Hugging Face, com bateria fixa e gabarito determinístico.
11. **Escala do dossiê:** perfilamento offline + recuperador de schema (RAG sobre catálogo) + fusão com governança de dados (seção 11).
12. **Seleção multiplataforma de modelos** por papel, otimizando custo×performance, com context caching.
13. **Execução endurecida:** réplica de leitura, timeout de consulta e guardrail de custo com EXPLAIN.
14. **Paralelização** de passos independentes do plano para reduzir latência.
15. **Exibição de tokens por chamada na UI** (já visíveis no LangSmith; exigiria include_raw no structured output).
16. **Replay de sessões** a partir do trace persistido, e métricas de acerto por tipo de pergunta.

25. Encerramento

Este sistema foi desenhado em torno de uma única ideia, aplicada com disciplina: **dar agência ao LLM onde há incerteza e manter determinismo onde ela é perigosa**. Interpretar a pergunta, escrever o SQL, julgar a resposta e desenhar o gráfico são tarefas de incerteza — e ali os agentes têm liberdade. Rotear, validar, executar e conter são mecânica — e ali o código não improvisa.

O resultado cumpre as quatro capacidades do enunciado e vai além delas em segurança, transparência e robustez, com uma metodologia que privilegiou a especificação antes do código, o teste do que é vulnerável e a honestidade técnica a cada decisão — inclusive ao admitir os limites do desenho e ao mostrar o caminho de escala. É a mesma engenharia que conduziu o Desafio 2, agora aplicada ao problema inverso.

Os agentes decidem o quê e o se; o grafo só garante o onde e o até quando.

Leandro Henrique da Silva

Engenheiro de Controle e Automação

+55 (47) 9.8906-7677 · dasilva.leandrohenrique1991@gmail.com